

Advanced JavaScript exercises

Instructions

Use VS Code and a browser for these exercises.

For exercises that contain multiple files:

- Create the files using the exact names shown.
- Keep the files in the same folder unless the exercise shows another structure.
- Open `index.html` through Live Server.
- Do not open browser module exercises directly through `file://`.

Complete each exercise independently unless it refers to another part of the same exercise.

Section A: DOM and browser interaction

Exercise 1: Update the page after a button click

Create these files:

```
exercise-01/  
├─ index.html  
└─ script.js
```

`index.html`

```
<!DOCTYPE html>  
<html lang="en">  
  <head>  
    <meta charset="UTF-8">  
    <meta  
      name="viewport"  
      content="width=device-width, initial-scale=1.0"  
    >  
    <title>Booking status</title>  
  </head>  
  <body>  
    <h1>Workshop booking</h1>  
  
    <p id="booking-status">  
      No booking has been made  
    </p>  
  
    <button id="confirm-booking">  
      Confirm booking
```

```
</button>

<script src="./script.js"></script>
</body>
</html>
```

Write the JavaScript required to:

1. Select the status paragraph using `getElementById`.
2. Select the button using `querySelector`.
3. Create a named handler function.
4. Change the paragraph to `Booking confirmed` after the button is clicked.
5. Register the handler without calling it immediately.

Exercise 2: Compare `event.target` and `event.currentTarget`

Create these files:

```
exercise-02/
├─ index.html
└─ script.js
```

`index.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta
    name="viewport"
    content="width=device-width, initial-scale=1.0"
  >
  <title>Delivery action</title>
</head>
<body>
  <button
    id="delivery-button"
    data-action="dispatch"
  >
    <span>Dispatch package</span>
  </button>

  <p id="event-result"></p>

  <script src="./script.js"></script>
</body>
</html>
```

script.js

```
const deliveryButton =
  document.querySelector("#delivery-button");

const eventResult =
  document.querySelector("#event-result");

deliveryButton.addEventListener(
  "click",
  event => {
    eventResult.textContent =
      `${event.target.tagName} / `
      + `${event.currentTarget.tagName}`;
  }
);
```

Answer these questions before running the page:

1. What will be displayed when the user clicks the words inside the `<span>`?
2. Which property refers to the element where the click began?
3. Which property refers to the element that owns the listener?

Then change the handler so that it displays:

Action: dispatch

Read the `data-action` value from the element that owns the listener.

Exercise 3: Add and remove notices

Create these files:

```
exercise-03/
├─ index.html
└─ script.js
```

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta
    name="viewport"
    content="width=device-width, initial-scale=1.0"
```

```

    >
    <title>Notice board</title>
</head>
<body>
  <h1>Notice board</h1>

  <p id="notice-status">
    No notices added
  </p>

  <button id="add-notice">
    Add notice
  </button>

  <button id="remove-notice">
    Remove latest
  </button>

  <ul id="notice-list"></ul>

  <script src="./script.js"></script>
</body>
</html>

```

Write the JavaScript required to produce this behaviour:

- Each click on **Add notice** creates a new `<li>`.
- The first item contains `Notice 1`, the second contains `Notice 2`, and so on.
- The new item is appended to the list.
- The status displays `Notice added`.
- **Remove latest** removes the final `<li>`.
- The status displays `Latest notice removed`.
- When the list is empty, clicking **Remove latest** displays `No notices to remove`.
- No error should appear when the list is empty.

Use named handler functions for both buttons.

Exercise 4: Choose between `textContent` and `innerHTML`

Create these files:

```

exercise-04/
├─ index.html
└─ script.js

```

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta
    name="viewport"
    content="width=device-width, initial-scale=1.0"
  >
  <title>Message preview</title>
</head>
<body>
  <label for="message-input">
    Message
  </label>

  <input
    id="message-input"
    value="<strong>Important</strong>"
  >

  <button id="show-message">
    Show message
  </button>

  <p id="message-preview"></p>

  <section id="trusted-summary"></section>

  <script src="./script.js"></script>
</body>
</html>
```

Complete both tasks.

User-entered message

When the button is clicked, display the input value exactly as the user entered it.

The page should show:

Important

The browser must not create a `<strong>` element from the input.

Trusted summary

Display this prepared markup inside `trusted-summary`:

```
<h2>Registration complete</h2>
<p>Your seat has been reserved.</p>
```

Choose `textContent` or `innerHTML` separately for each task. Briefly explain both choices.

Exercise 5: Display Browser Object Model information

Create these files:

```
exercise-05/
├─ index.html
└─ script.js
```

`index.html`

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta
    name="viewport"
    content="width=device-width, initial-scale=1.0"
  >
  <title>Browser details</title>
</head>
<body>
  <button id="show-details">
    Show browser details
  </button>

  <p>
    Current address:
    <span id="page-address">Not loaded</span>
  </p>

  <p>
    Browser language:
    <span id="browser-language">Not loaded</span>
  </p>

  <p>
    Screen width:
    <span id="screen-width">Not loaded</span>
  </p>

  <script src="./script.js"></script>
```

```
</body>
</html>
```

When the button is clicked, display:

- The current page address from `location`
- The browser language from `navigator`
- The screen width from `screen`

Then identify which parts of the solution use:

1. The DOM
2. The Browser Object Model

Section B: Modern JavaScript syntax

Exercise 6: Extract values with destructuring

Start with this data:

```
const booking = {
  bookingId: "BK-204",
  guestName: "Ira Sen",
  status: "Confirmed",
  preferences: [
    "Window seat",
    "Vegetarian meal",
    "Email updates"
  ]
};
```

Use object destructuring to create these bindings:

```
bookingId
guestName
status
```

Use array destructuring to create:

```
primaryPreference
otherPreferences
```

`primaryPreference` should contain the first preference. `otherPreferences` should contain the remaining preferences.

Use the extracted values to print:

BK-204 | Ira Sen | Confirmed
Primary preference: Window seat
Other preferences: 2

Exercise 7: Update data without changing the original

Start with this object:

```
const product = {  
  id: "PR-108",  
  name: "Wireless mouse",  
  stock: 4,  
  tags: [  
    "electronics",  
    "accessories"  
  ]  
};
```

Create a new object named `updatedProduct`.

Apply these changes only to the new object:

- Change `stock` to `7`.
- Add `"featured"` to the tags.
- Keep every other property unchanged.

After creating the new object, print:

```
console.log(product.stock);  
console.log(product.tags);  
console.log(updatedProduct.stock);  
console.log(updatedProduct.tags);
```

The original object and its original tags must remain unchanged.

Briefly explain why spreading only `product` is not enough to protect the original `tags` array when adding another tag.

Exercise 8: Collect extra arguments with a rest parameter

Write a function named `createActivitySummary`.

The first parameter is the activity title. Every remaining argument is a learner name.

These calls should work:

```

createActivitySummary(
  "DOM practice",
  "Aman",
  "Jaya",
  "Omar"
);

createActivitySummary(
  "Promise review",
  "Leena"
);

```

The first call should return:

```

DOM practice: 3 learners
Aman, Jaya, Omar

```

The second should return:

```

Promise review: 1 learner
Leena

```

Use a rest parameter to collect the learner names into an array.

Exercise 9: Preserve `this` inside a delayed callback

Read this code:

```

const reminder = {
  message: "Submit the assignment",

  start() {
    setTimeout(function () {
      console.log(this.message);
    }, 500);
  }
};

reminder.start();

```

The delayed callback does not reliably read `message` from the `reminder` object.

Correct the code by changing only the callback passed to `setTimeout`.

Then explain:

1. What `this` refers to when `start()` is called as `reminder.start()`
2. Why the corrected callback can use the same `this`
3. Why changing `start()` itself into an arrow function would not be a suitable correction

Exercise 10: Refactor a formatter

Refactor this function using:

- A `const` variable
- An arrow function
- Object destructuring
- A template literal

```
function formatSession(session) {  
  return session.title  
    + " starts at "  
    + session.startTime  
    + " in "  
    + session.room;  
}
```

Test the refactored function with:

```
const session = {  
  title: "JavaScript modules",  
  startTime: "10:00",  
  room: "Lab 3"  
};
```

Expected result:

```
JavaScript modules starts at 10:00 in Lab 3
```

Section C: JavaScript modules

Exercise 11: Repair a browser module

Create these files:

```
exercise-11/  
├─ index.html
```

```
├─ app.js
└─ pricing.js
```

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta
    name="viewport"
    content="width=device-width, initial-scale=1.0"
  >
  <title>Order total</title>
</head>
<body>
  <p id="order-total"></p>

  <script src="./app.js"></script>
</body>
</html>
```

pricing.js

```
export function calculateSubtotal(
  price,
  quantity
) {
  return price * quantity;
}
```

app.js

```
import {
  calculateTotal
} from "pricing.js";

const orderTotal =
  document.querySelector("#order-total");

orderTotal.textContent =
  calculateTotal(250, 3);
```

The page does not display the expected total.

Find and correct all module-related problems.

The completed page should display:

750

Exercise 12: Split one script by responsibility

The following code currently exists in one file:

```
const courses = [
  {
    id: 1,
    title: "JavaScript fundamentals"
  },
  {
    id: 2,
    title: "Advanced JavaScript"
  }
];

function findCourse(courseId) {
  return courses.find(
    course => course.id === courseId
  );
}

const courseTitle =
  document.querySelector("#course-title");

const selectedCourse =
  findCourse(2);

courseTitle.textContent =
  selectedCourse.title;
```

Split it into this structure:

```
exercise-12/
├─ index.html
├─ app.js
├─ course-data.js
└─ course-utils.js
```

Apply these responsibilities.

`course-data.js`

Store and export the `courses` array.

`course-utils.js`

Export `findCourse`. The function should receive both the course list and the course ID. It should not import the course data itself.

`app.js`

Import the data and function, find course `2`, and display its title.

`index.html`

Contain this element:

```
<h1 id="course-title"></h1>
```

Load `app.js` as the browser module entry point.

The page should display:

Advanced JavaScript

Exercise 13: Import only the public functions

Create these files:

```
exercise-13/  
├─ index.html  
├─ app.js  
└─ formatters.js
```

`index.html`

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
  <meta charset="UTF-8">  
  <meta  
    name="viewport"  
    content="width=device-width, initial-scale=1.0"  
  >  
  <title>Formatting</title>  
</head>  
<body>  
  <script  
    type="module"  
    src="./app.js"
```

```
></script>
</body>
</html>
```

formatters.js

```
function capitalize(value) {
  return value.charAt(0).toUpperCase()
    + value.slice(1);
}

export function formatName(name) {
  return capitalize(name);
}

export function formatStatus(status) {
  return `Status: ${capitalize(status)}`;
}
```

app.js

Import the two exported functions and print:

```
Meera
Status: Confirmed
```

Use these input values:

```
"meera"
"confirmed"
```

Then answer:

1. Why can `app.js` import `formatName` and `formatStatus`?
2. Why can it not use a named import for `capitalize`?
3. Why must the imported names match the exported names?

Section D: Asynchronous JavaScript

Exercise 14: Predict the execution order

Read this code without running it:

```
console.log("Start");
```

```
setTimeout(() => {  
  console.log("Timer finished");  
}, 0);  
  
console.log("End");
```

Write the exact order of the three messages.

Then explain why a delay of `0` does not make the timer callback run before `console.log("End")`.

Exercise 15: Pass a callback instead of calling it immediately

Read this code:

```
function showStatus(message) {  
  console.log(message);  
}  
  
setTimeout(  
  showStatus("Ready"),  
  500  
);
```

The message is printed immediately instead of after the delay.

Correct the `setTimeout` call so that `Ready` is printed after approximately 500 milliseconds.

Then explain the difference between:

`showStatus`

and:

`showStatus()`

in this situation.

Exercise 16: Report success or failure through callbacks

Complete this function:

```
function loadSeatAvailability(  
  shouldFail,  
  onSuccess,  
  onError
```

```
) {  
  // Complete the function.  
}
```

Apply these rules:

- Wait 500 milliseconds before producing a result.
- When `shouldFail` is `false`, call `onSuccess` with this object:

```
{  
  availableSeats: 6  
}
```

- When `shouldFail` is `true`, call `onError` with:

```
Seat information could not be loaded
```

Test the function once for success and once for failure.

The success callback should print:

```
Available seats: 6
```

The error callback should print the supplied error message.

Exercise 17: Trace dependent callbacks

Assume these prepared functions run asynchronously:

```
function loadOrder(orderId, callback) {  
  setTimeout(() => {  
    callback({  
      id: orderId,  
      customerId: 42,  
      total: 850  
    });  
  }, 300);  
}  
  
function loadCustomer(  
  customerId,  
  callback  
) {  
  setTimeout(() => {  
    callback({  
      id: customerId,
```

```

        name: "Nikhil"
    });
}, 300);
}

```

The functions are used like this:

```

loadOrder("OR-108", order => {
  loadCustomer(
    order.customerId,
    customer => {
      console.log(
        `${customer.name}: `
        + `${order.total}`
      );
    }
  );
});

```

Answer these questions:

1. Which operation starts first?
2. Why can `loadCustomer` not use `order.customerId` before the order callback runs?
3. What message is printed?
4. What would make this structure harder to read if several more dependent operations were added?

Then extract the final display code into a named function called `showOrderSummary`. Keep the two asynchronous calls working in the same order.

Exercise 18: Create and consume a Promise

Write a function named `checkRoomAvailability`.

The function receives:

```

roomNumber
shouldFail

```

It must return a Promise.

After 500 milliseconds:

- Resolve with this object when `shouldFail` is `false`:

```

{
  roomNumber,

```

```
    available: true
  }
```

- Reject with an `Error` containing this message when `shouldFail` is `true`:

```
Room information could not be loaded
```

Use `.then()` to print:

```
Room 204 is available
```

Use `.catch()` to print the error message.

Test the success and failure cases separately.

Exercise 19: Return a value to the next Promise stage

Use this prepared function:

```
function loadProfile() {
  return Promise.resolve({
    name: "Meera",
    email: "meera@example.com"
  });
}
```

The second `.then()` callback currently receives `undefined`:

```
loadProfile()
  .then(profile => {
    const updatedProfile = {
      ...profile,
      theme: "Dark"
    };
  })
  .then(updatedProfile => {
    console.log(
      updatedProfile.theme
    );
  })
  .catch(error => {
    console.log(error.message);
  });
```

Correct the chain so that the second callback receives the updated profile.

Then answer:

1. What value does the first `.then()` callback currently return?
2. Why does the next callback receive that value?
3. Where will an error from either `.then()` callback be handled?

Exercise 20: Rewrite a Promise chain with `async` and `await`

Start with this Promise-based service:

```
function loadBooking(shouldFail) {
  return new Promise(
    (resolve, reject) => {
      setTimeout(() => {
        if (shouldFail) {
          reject(
            new Error(
              "Booking could not be loaded"
            )
          );
          return;
        }

        resolve({
          bookingId: "BK-501",
          guestName: "Rohan",
          status: "Confirmed"
        });
      }, 500);
    }
  );
}
```

The service is currently consumed like this:

```
loadBooking(false)
  .then(booking => {
    console.log(
      `${booking.bookingId}: `
      + `${booking.status}`
    );
  })
  .catch(error => {
    console.log(error.message);
  });
```

Rewrite the consuming code as an asynchronous function named `showBooking`.

The function should:

- Receive `shouldFail`
- Wait for `loadBooking`
- Destructure `bookingId` and `status`
- Print the successful result
- Handle a rejected Promise with `try` and `catch`

Call the function once with `false` and once with `true`. Test the calls separately.

Then answer:

1. What does an `async` function return?
2. What does `await` receive when the Promise is fulfilled?
3. Where does execution move when the awaited Promise rejects?

Final exercise: Build a workshop availability viewer

Create this file structure:

```
workshop-viewer/  
├─ index.html  
├─ main.js  
├─ workshop-service.js  
└─ ui.js
```

Run the page through Live Server.

`index.html`

Use this prepared page:

```
<!DOCTYPE html>  
<html lang="en">  
<head>  
  <meta charset="UTF-8">  
  <meta  
    name="viewport"  
    content="width=device-width, initial-scale=1.0"  
  >  
  <title>Workshop viewer</title>  
</head>  
<body>  
  <h1>Workshop availability</h1>  
  
  <button id="load-workshop">
```

```

    Load workshop
  </button>

  <button id="simulate-error">
    Simulate error
  </button>

  <p id="workshop-status">
    Select an action
  </p>

  <section id="workshop-details"></section>

  <script
    type="module"
    src="./main.js"
  ></script>
</body>
</html>

```

workshop-service.js

Create and export a function named `loadWorkshop`.

It should receive `shouldFail` and return a Promise.

After 600 milliseconds:

- Reject with an `Error` containing `Workshop could not be loaded` when `shouldFail` is `true`.
- Otherwise resolve with:

```

{
  id: "WS-208",
  title: "JavaScript practice lab",
  facilitator: "Neha",
  status: "Scheduled",
  topics: [
    "DOM",
    "Modules",
    "Promises"
  ]
}

```

ui.js

Export these functions:

```
showStatus
showWorkshop
clearWorkshop
```

showStatus

Receive a message and display it inside `workshop-status`.

clearWorkshop

Remove all content from `workshop-details`.

showWorkshop

Receive a workshop object.

Use destructuring to read:

```
id
title
facilitator
status
topics
```

Create an updated copy of the workshop whose status is `"Open"` without changing the original object.

Display this trusted markup inside `workshop-details`:

```
WS-208
JavaScript practice lab
Facilitator: Neha
Status: Open
Topics: DOM, Modules, Promises
```

Use headings or paragraphs so that each value is readable.

main.js

Import the service and UI functions.

Create an asynchronous function named `handleWorkshopLoad`.

It should:

1. Receive `shouldFail`.
2. Clear the previous workshop.

3. Display `Loading workshop...`.
4. Await `loadWorkshop(shouldFail)` inside `try`.
5. Display the workshop when loading succeeds.
6. Display `Workshop loaded` after rendering.
7. Clear the workshop and display the error message when loading fails.

Register both button listeners:

- **Load workshop** calls the function with `false`.
- **Simulate error** calls the function with `true`.

Completion checks

- The page runs through Live Server.
- Every import uses the correct exported name and relative `.js` path.
- The loading message appears before the result.
- The successful action displays the workshop.
- The displayed status is `Open`.
- The original service object is not directly modified.
- The failure action clears old workshop content.
- The failure message is displayed.
- No unhandled Promise rejection appears in the console.